Exercise No: 4	Monitoring and Alerting with CloudTrail and CloudWatch
Date of Exercise	25.09.2026

AIM:
To understand and implement monitoring and alerting in Amazon Web Services (AWS) using AWS CloudTrail, Amazon CloudWatch, Amazon EventBridge, and Amazon Simple Notification Service (SNS). To monitor API activity, configure email notifications for security group modifications, create metric filters and alarms for failed console login attempts, and analyze CloudTrail logs using CloudWatch Logs Insights.

DESCRIPTION:
This lab focused on implementing monitoring and alerting mechanisms in Amazon Web Services (AWS) using AWS CloudTrail, Amazon CloudWatch, Amazon EventBridge and Amazon Simple Notification Service (SNS). CloudTrail event history was examined to understand recorded API activity. An attempt was made to create a new CloudTrail trail with CloudWatch Logs integration, but the operation could not be completed due to the permissions available in the lab environment. The preconfigured trail was therefore examined instead.

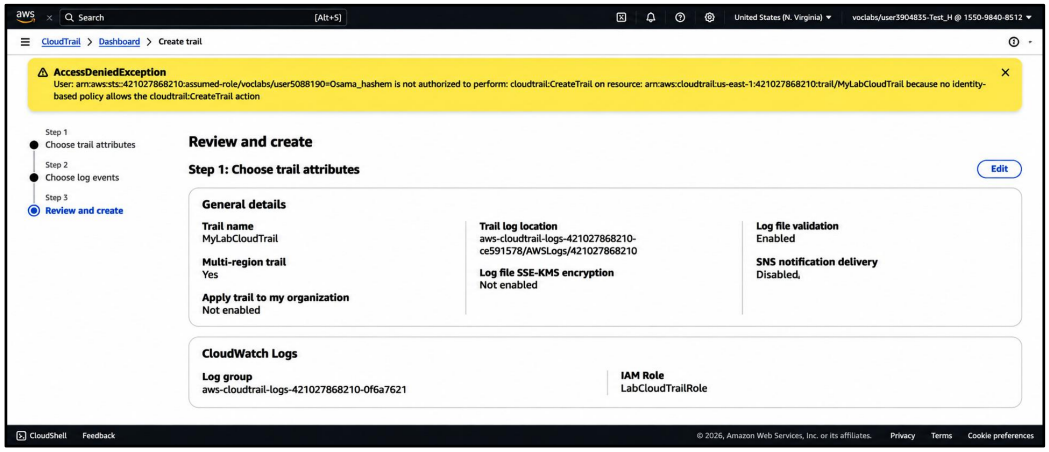
The activities also involved creating an SNS topic and confirming an email subscription to receive security notifications. An EventBridge rule was configured to detect security group modifications and forward event details through SNS. Additionally, a CloudWatch metric filter and alarm were created to detect repeated failed console login attempts. Finally, CloudWatch Logs Insights was used to query and analyse CloudTrail records related to unsuccessful login activity.

PROCEDURE:

Task 1: Creating a CloudTrail Trail with CloudWatch Logs Enabled
The CloudTrail Event history was examined by filtering events generated by cloudformation.amazonaws.com. The CreateStack event was selected to review details such as user identity, event time, and AWS Region. An attempt was made to configure a new trail named MyLabCloudTrail with an S3 bucket, CloudWatch Logs integration, a new log group, the existing LabCloudTrailRole, and read and write management events. However, the required permissions to create the trail were unavailable in the lab environment. Therefore, the creation process was cancelled, and the preconfigured LabCloudTrail was examined to understand its logging configuration.

Result: CloudTrail event history was successfully inspected, and the configuration of the existing trail was reviewed. The new trail could not be created because of the lab's permission restrictions.

OUTPUT:

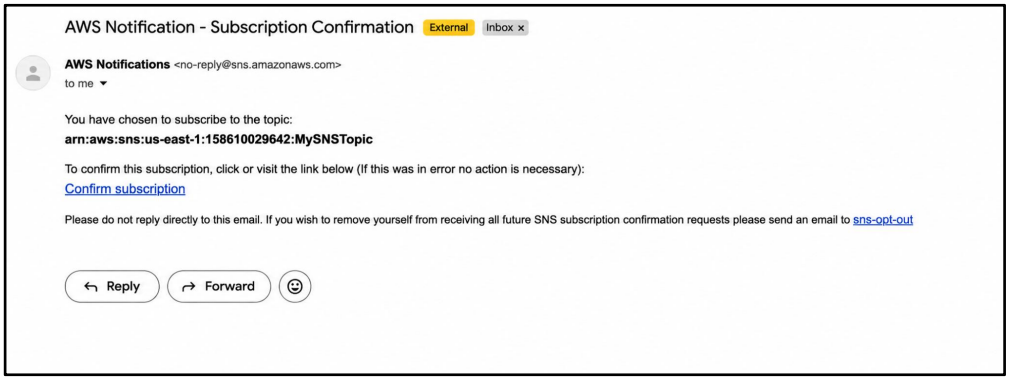


Task 2: Creating an SNS Topic and Subscribing to It

Amazon Simple Notification Service (SNS) was opened, and a Standard topic named MySNSTopic was created. The topic access policy was configured according to the lab instructions to allow publishing and subscribing. An email subscription was then added using the required email address. The subscription confirmation message was opened from the AWS Notifications email, and the confirmation link was selected to activate the subscription.

Result: The SNS topic MySNSTopic was created, and the email subscription was confirmed successfully. The topic was ready to deliver notifications to the subscribed email address.

OUTPUT:



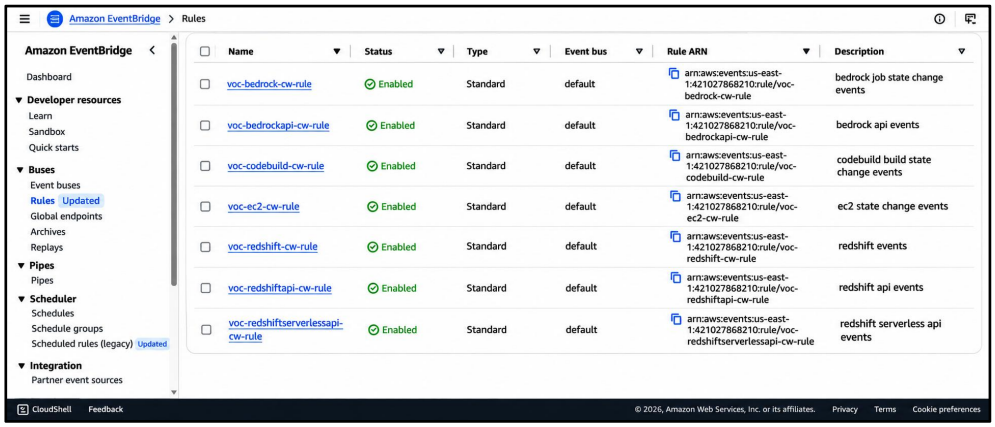
Task 3: Creating an EventBridge Rule to Monitor Security Groups

Amazon EventBridge was opened, and a rule named MonitorSecurityGroups was created on the default event bus. A custom event pattern was configured to detect EC2 security group changes generated through CloudTrail. The pattern matched events from ec2.amazonaws.com with the event names AuthorizeSecurityGroupIngress and ModifyNetworkInterfaceAttribute. The SNS topic MySNSTopic was selected as the target, and an input transformer was configured to include relevant event details such as the security group ID, API event name, event time, and event details. To test the rule, an inbound SSH rule

allowing access from Anywhere-IPv4 was added to LabSecurityGroup. The corresponding AuthorizeSecurityGroupIngress event was then examined in CloudTrail, and the email notification was checked.

Result: The EventBridge rule detected the security group modification event and forwarded the notification to MySNSTopic. The email notification confirmed that the monitoring configuration was functioning as expected.

OUTPUT:

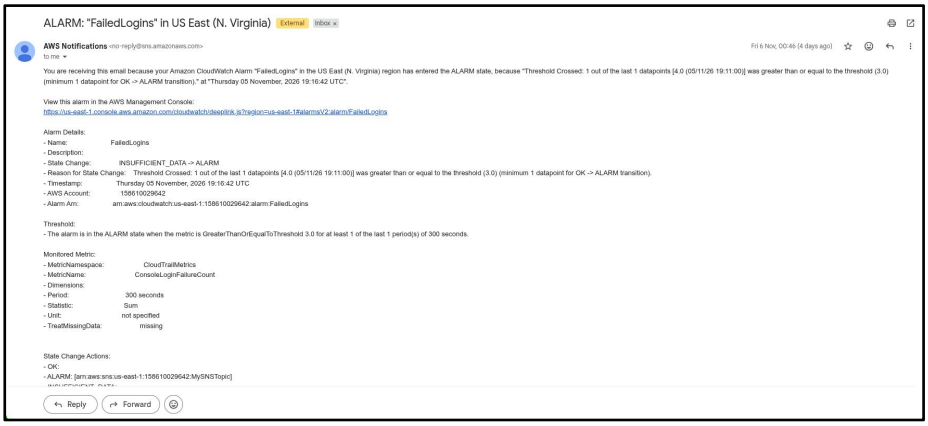


Task 4: Creating a CloudWatch Alarm Based on a Metric Filter

Amazon CloudWatch Logs was opened, and a metric filter was created for the CloudTrailLogGroup using the filter pattern Ⓔ (Ⓣ.◯␣◯∅N∠Ⓜ◯ = C□∅Ⓢ□Ⓛ◯L□/◯/=∅) && (Ⓣ.◯◎◎□◎M◯ⓈⓈ∠/◯ = "F∠=Ⓛ◯⌓ ∠—/ /◯∅/=▱∠/=□∅") Ⓡ. The filter was named ConsoleLoginErrors and configured with the namespace CloudTrailMetrics, metric name ConsoleLoginFailureCount, and metric value 1. A CloudWatch alarm named FailedLogins was then created to monitor this metric. The alarm was configured to trigger when the metric value reached or exceeded 3 within a 5-minute period, with MySNSTopic selected for notifications. To test the configuration, multiple unsuccessful console login attempts were made using the IAM user test and an incorrect password. The resulting metric activity and alarm state were examined, and the email notification was checked.

Result: The metric filter detected failed console login attempts, and the FailedLogins alarm entered the In alarm state after the configured threshold was reached. An email notification was delivered through the SNS topic.

OUTPUT:

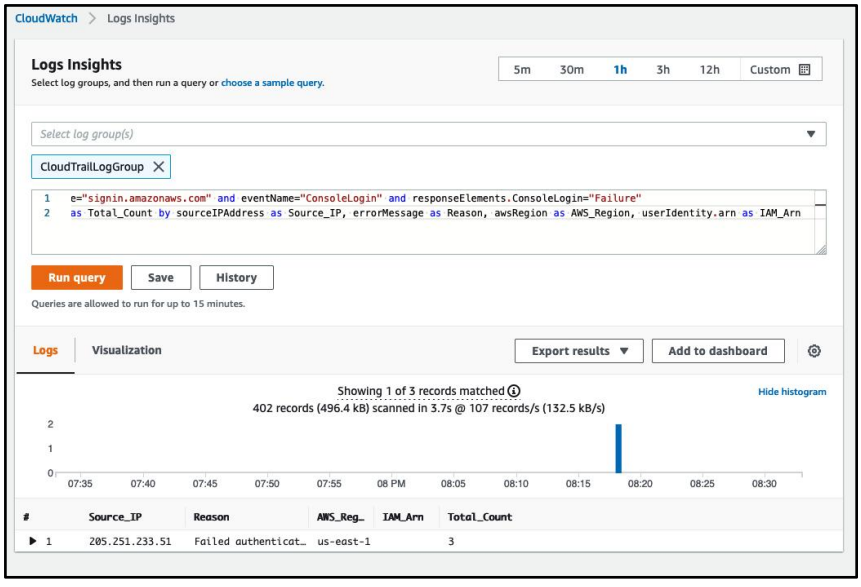


Task 5: Querying CloudTrail Logs by Using CloudWatch Logs Insights

Amazon CloudWatch Logs Insights was opened, and the CloudTrailLogGroup was selected as the data source. A query was entered to identify failed AWS console login events by filtering for signin.amazonaws.com as the event source, ConsoleLogin as the event name, and Failure as the login response. The query was further configured to group the results by source IP address, error message, AWS Region, and IAM user ARN, while calculating the total number of matching events. The query was executed, and the resulting records were examined to understand the failed login activity.

Result: CloudTrail log data was successfully queried using CloudWatch Logs Insights. The results provided information about failed console login attempts, including their source IP addresses, failure reasons, AWS Regions, and associated IAM identities.

OUTPUT:



BADGE:



PRESENTATION LINK:

[Insert YouTube / presentation link]

RESULT:

The lab demonstrated how AWS monitoring services can be integrated to detect and report security-related activities. CloudTrail event history and the preconfigured trail were examined, while EventBridge was configured to detect security group modifications and send notifications through SNS. A CloudWatch metric filter and alarm were also created to identify repeated failed console login attempts. CloudWatch Logs Insights was used to analyse the recorded events. The new CloudTrail trail could not be created due to lab permission restrictions.